`scpn-quantum-control`: reproducible Kuramoto–XY quantum-control workflows with Rust acceleration and hardware artefact packaging

Miroslav Sotek
ANULUM, Marbach SG, Switzerland
ORCID: 0009-0009-3560-0851

5 May 2026

Summary

`scpn-quantum-control` is a research software package for constructing, simulating, benchmarking, and packaging quantum-control experiments derived from heterogeneous Kuramoto oscillator networks and their corresponding XY spin Hamiltonians. The package maps coupling matrices and natural frequencies to Qiskit Hamiltonians, variational ansatze, Trotter circuits, simulator workflows, and IBM Quantum hardware execution records. It combines a Python/Qiskit front end with Rust/PyO3 acceleration for selected hot-path kernels and includes scripts that regenerate benchmark tables into JSON and CSV artefacts.

The package has been used to produce a companion IBM Heron r2 hardware preprint and public validation package on parity-sector and excitation-number correlated decoherence asymmetry in heterogeneous XY circuits. The same repository now includes generated artefacts for Rust kernel timings, topology-informed ansatz construction, VQE comparisons, multi-language coupling-matrix benchmarks, cross-machine CPU runs, and Vertex T4 GPU batched expectation benchmarks.

Statement of need

NISQ quantum-simulation studies often contain a gap between the mathematical model and the artefacts needed to verify a hardware claim. A complete workflow must specify how the coupling matrix was generated, how the Hamiltonian was constructed, how circuits were compiled, which classical baselines were used, which hardware jobs were submitted, and how raw-count dictionaries were analysed. Without this chain, small changes in ansatz topology, transpilation, readout processing, or benchmark implementation can dominate the reported result.

`scpn-quantum-control` addresses this gap for Kuramoto–XY and SCPN phase dynamics experiments. Its target users are quantum software researchers, NISQ experimentalists, and computational physicists who need reproducible pipelines connecting oscillator-network models to circuits, simulators, hardware runs, and published artefacts. The software is not presented as a quantum-advantage engine; rather, it is an auditable workflow for small- and medium-scale hardware experiments where honest classical baselines and raw data provenance are essential.

State of the field

Qiskit provides the general quantum-circuit, operator, transpilation, and execution infrastructure used by the package. Variational workflows such as VQE and hardware-efficient ansatze are also well established. Existing tools, however, do not provide an end-to-end Kuramoto–XY workflow that joins oscillator coupling construction, topology-informed ansatz generation, Rust-accelerated kernels, IBM raw-count packaging, and generated performance artefacts in one repository.

The contribution of `scpn-quantum-control` is therefore not to replace Qiskit, but to specialise it for a research programme in heterogeneous oscillator networks, XY Hamiltonians, and symmetry-aware NISQ validation. The package keeps Qiskit as the circuit and operator layer while adding domain-specific Hamiltonian construction, experiment manifests, integrity hashing, and benchmark harnesses.

Quickstart

The core workflow constructs a coupling matrix, maps it to a Hamiltonian, and generates a topology-informed ansatz whose entangling graph follows the non-zero coupling structure:

```
from scpn_quantum_control.bridge.knm_hamiltonian import (
    OMEGA_N_16,
    build_knm_paper27,
    knm_to_ansatz,
    knm_to_hamiltonian,
)

n = 4
k_matrix = build_knm_paper27(n)
hamiltonian = knm_to_hamiltonian(k_matrix, OMEGA_N_16[:n])
ansatz = knm_to_ansatz(k_matrix, reps=2)
```

The same artefacts can then be passed to VQE routines, simulator workflows, hardware runners, or benchmark scripts. For hardware execution and packaging, the repository provides runner and manifest utilities that archive job identifiers, raw counts, circuit metadata, and integrity hashes.

Software design

The architecture separates orchestration from hot kernels. Python modules provide the public research interface: coupling-matrix construction, Hamiltonian conversion, ansatz generation, VQE workflows, IBM execution packaging, and analysis scripts. Rust/PyO3 modules provide deterministic low-overhead kernels for selected performance-sensitive paths. This design keeps the package usable by Python-based quantum researchers while allowing kernel-level optimisation where repeated allocation or object dispatch would otherwise dominate.

The methods-paper benchmark harnesses follow an artefact-first rule: numerical tables are regenerated from scripts and committed as JSON/CSV summaries rather than copied manually into prose. The current generated artefacts include: Rust/core kernel summaries, ansatz construction summaries, VQE aggregate tables, multi-language coupling-matrix comparisons, local/ML350/Vertex CPU runs, and a Vertex T4 batched state-vector expectation benchmark. The

GPU benchmark is intentionally separated from the Rust CPU benchmarks because it targets batched dense linear algebra for classical validation, not scalar coupling-matrix construction.

The main public workflow modules include `bridge/knm_hamiltonian.py` for Kuramoto–XY matrix and Hamiltonian conversion, `control/structured_ansatz.py` for topology-informed circuit construction, `phase/phase_vqe.py` for VQE-style workflows, and hardware runner utilities for simulator/IBM execution. The benchmark harnesses under `scripts/` regenerate the paper artefacts rather than requiring manual table editing.

Research impact statement

The companion hardware study provides raw-count archives, job identifiers, SHA256 integrity hashes, reproduction commands, and bounded statistical claims. The benchmark artefacts generated on 5 May 2026 show that the project is also usable as a reproducible methods platform: at $n = 16$, the self-contained coupling-matrix benchmark reports Rust/Python median speedups of 44.1x on the local workstation, 15.5x on the ML350 server, and 18.1x on Vertex AI CPU. A separate Vertex Tesla T4 run reports CUDA median times of 0.176–0.905 ms for batched expectation evaluation at $n = 8$–11. These results support a hybrid software strategy in which Rust handles low-latency CPU kernels, GPUs handle batched dense expectation evaluation, and QPU time is reserved for hardware-noise questions that classical simulation cannot answer directly.

The package fills a narrow but reusable niche: transparent Kuramoto–XY quantum simulation workflows with benchmarked software paths and hardware artefact lineage. This makes it useful both as research infrastructure for future SCPN experiments and as an example of how small-N NISQ studies can publish their software, raw data, and performance claims without relying on unverifiable manual tables.

AI usage disclosure

AI-assisted tools were used for drafting and editing support. Numerical claims, artefact provenance, scientific framing, authorship, and final responsibility were verified and retained by the author.

References

1. M. Sotek, *`scpn-quantum-control`: Quantum-Native SCPN Phase Dynamics and Control*, version 0.9.6, Zenodo record 18821929 (2026).

2. M. Sotek, "Replicated hardware observation of parity-sector and excitation-number correlated decoherence asymmetry in the XY Hamiltonian," preprint manuscript (2026).

3. M. Sotek, *DLA parity hardware validation package for heterogeneous Kuramoto–XY circuits*, Zenodo record 19098792 (2026).

4. Qiskit contributors, *Qiskit: An Open-source Framework for Quantum Computing*, Zenodo record 2573505.

5. A. Peruzzo et al., "A variational eigenvalue solver on a photonic quantum processor," *Nature Communications* 5, 4213 (2014), doi:10.1038/ncomms5213.

6. A. Kandala et al., "Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets," *Nature* 549, 242–246 (2017), doi:10.1038/nature23879.

7. Y. Kuramoto, *Chemical Oscillations, Waves, and Turbulence*, Springer (1984), doi:10.1007/978-3-642-69689-3.